Script-Controlled ACL – Restrict Record Access Based on Field Value

Phase 7 -Sample Project Documentation

Project Description:

This project demonstrates how to implement a script-controlled Access Control List (ACL) to restrict users from viewing records unless they meet specific conditions. In this scenario, only users belonging to the EEE branch are allowed to view records where the Branch field is set to EEE, while administrators retain full access.

Timer: 3 hrs

Scenarios

Scenario 1: An EEE User with the bb1 role opens the Student Records list. The user can view only EEE branch records.

Scenario 2: A user without the required role attempts to open the Student Records list. Access to the records is denied.

Scenario 3: An Administrator opens the Student Records list. The administrator can view all records regardless of branch.

Scenario 4: Users are given bb1, bb2, bb3 and bb4 progressively to verify read, create, write and delete permissions.

Technical Architecture

Description:

The project uses a ServiceNow-based access-control architecture. The ServiceNow instance provides the user, role, table, record and ACL management environment. The custom u_institution_details table stores student-related records. ACL rules evaluate the requested operation, required role and, for the READ ACL, the Branch field condition. A script-controlled ACL grants administrators full access and permits the required role to pass the configured security rule.

Main components:

- ServiceNow Instance – execution and configuration platform.
- Users – EEE User and Administrator/test users.
- Custom Roles – bb1, bb2, bb3 and bb4.
- Custom Table – u_institution_details (Institution Details).
- Record Fields – Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number and Description.
- Access Control Lists – READ, CREATE, WRITE and DELETE.

Note: The source project does not specify a database ER diagram; the project is implemented through ServiceNow table and ACL configuration.

Pre-requisites:

- ServiceNow instance with administrator access.

- Knowledge of User Administration and Roles.
- Knowledge of System Definition → Tables.
- Knowledge of System Security → Access Control (ACL).
- Ability to elevate the Security_admin role.
- Understanding of ServiceNow impersonation and list access.

Project Workflow

Milestone 1: Creation of Users and Roles

- Activity 1.1: Create the EEE User.
- Activity 1.2: Create custom roles bb1, bb2, bb3 and bb4 and assign them to the EEE User.

Milestone 2: Table and Record Development

- Activity 2.1: Create the Institution Details custom table.
- Activity 2.2: Add the required fields and create records with ECE, EEE and CSE branch values.

Milestone 3: READ ACL Development

- Activity 3.1: Create and configure the script-controlled READ ACL.
- Activity 3.2: Verify role-based and branch-based visibility.

Milestone 4: CREATE, WRITE and DELETE ACL Development

- Activity 4.1: Configure the CREATE ACL.
- Activity 4.2: Configure the WRITE ACL.
- Activity 4.3: Configure the DELETE ACL.

Milestone 5: Testing and Verification

- Activity 5.1: Impersonate users and verify each ACL operation.
- Activity 5.2: Verify administrator access and cumulative permissions.

Milestone 6: Conclusion

Milestone 1: Creation of Users and Roles

In this milestone, the required ServiceNow users and custom roles are created. These identities are used to test the ACL rules and demonstrate different levels of access.

Activity 1.1: Create the EEE User

1. Log in to the ServiceNow instance with admin access.
2. Navigate to User Administration → Users.
3. Click New.
4. Enter User ID: EEE User.
5. Enter First name: EEE.
6. Enter Last name: User.
7. Enter Email: eeeuser@gmail.com.
8. Save the record.

The source project specifies the above EEE User details.

Activity 1.2: Create Custom Roles

Navigate to User Administration → Roles.

9. Click New.
10. Enter the role name as bb1.
11. Enter a brief role purpose in the Description field.
12. Click Submit.

Create the other roles using the same procedure:

- bb2 – custom role used for the CREATE ACL.
- bb3 – custom role used for the WRITE ACL.
- bb4 – custom role used for the DELETE ACL.

Assign all the roles to the EEE User as specified in the project.

Role progression used in the project:

- bb1 – READ access condition.
- bb1 + bb2 – READ plus CREATE capability.
- bb1 + bb2 + bb3 – READ, CREATE and WRITE capability.
- bb1 + bb2 + bb3 + bb4 – READ, CREATE, WRITE and DELETE capability.

Milestone 2: Table and Record Development

Activity 2.1: Create the Institution Details Table

13. In the Application Navigator, search for System Definition → Tables.
14. Click New.
15. Create a custom table with Label: Institution Details.
16. Set Name: u_institution_details.
17. Set Extends for: false.
18. Click Submit.

The custom table u_institution_details is the record set protected by the ACL rules.

Activity 2.2: Add Table Fields

Add the following fields:

- Student Roll Number – Auto Number
- Student Name – Reference – User
- Faculty Name – Reference – User
- Branch – Choice (ECE, EEE, CSE)
- Email – String
- Phone Number – String
- Description – Multi String

Activity 2.2: Create Test Records

After adding the fields, save the table.

19. Create multiple records in the Student Records table.
20. Use different branch values such as ECE, EEE and CSE.

The different Branch values are required to verify the field-based READ restriction.

Record-level security scenario:

- EEE records are the records intended to be visible to the EEE-role user under the configured READ condition.
- ECE and CSE records provide contrasting data for verification.
- Administrator access is retained for all records.

The source project uses the table name u_institution_details and refers to the Student Records list for verification.

Milestone 3: READ ACL Development

Activity 3.1: Creation of Access Control Lists (READ ACL)

21. In the Application Navigator, search for System Security → Access Control (ACL).
22. Elevate the Security_admin role.
23. Click New to create a new ACL.
24. Set Type: record.
25. Set Operation: read.
26. Set Name: u_institution_details.
27. Set Active: true.
28. Set Advanced: true.
29. In the Requires role related list, add bb1.
30. Add Data condition as Branch is EEE.

The READ ACL combines the configured role and Branch condition with an advanced script.

Activity 3.1: Script-Controlled READ ACL

In the Script field, add the following script from the project:

```
(function () {
  // Allow admin users full access
  if (gs.hasRole('admin')) {
    return true;
  }

  // Allow only EEE branch users to see EEE records
  if (gs.hasRole('bb1')){
    return true;
  }

  // Deny access for all others
```

```
    return false;
})();
```

31. Click Submit to save the ACL.

The script explicitly returns true for administrators, returns true when the user has the bb1 role, and returns false for all other users.

Activity 3.2: READ ACL Verification

Verify the READ ACL as specified in the source project:

32. Impersonate EEE users and open the Student Records list.
33. Copy the table name u_institution_details.list/LIST and paste it in the All menu.
34. Impersonate Admin and open the Student Records list.

Verify:

- User with bb1 role can view only EEE branch records.
- User without the role cannot view any records.
- Admin user can view all records regardless of branch.

This verification demonstrates how record-level access is controlled through the ACL configuration.

Milestone 4: CREATE ACL Development

Activity 4.1: Creation of Access Control Lists (CREATE ACL)

35. Click New to create a new ACL (Create).
36. Set Type: record.
37. Set Operation: create.
38. Set Name: u_institution_details.
39. Set Active: true.
40. In the Requires role related list, add bb2.
41. No Data condition is required for this ACL.
42. Save the ACL Record.

Verify:

- A user with bb1 and bb2 roles can view the permitted EEE branch records and use the New Button.

Milestone 4: WRITE ACL Development

Activity 4.2: Creation of Access Control Lists (WRITE ACL)

43. Click New to create a new ACL (Write).
44. Set Type: record.
45. Set Operation: Write.
46. Set Name: u_institution_details.
47. Set Active: true.
48. In the Requires role related list, add bb3.
49. No Data condition is required.
50. Save the ACL Record.

The WRITE ACL is used together with the previously configured READ and CREATE permissions.

Activity 4.2: WRITE ACL Verification

Verify:

- User with bb1, bb2 and bb3 roles can view and edit the EEE branch records along with the New Button.

This stage demonstrates the cumulative role-based permission model used by the project.

Access progression:

- bb1 provides the configured READ permission.
- bb2 adds CREATE permission.
- bb3 adds WRITE permission.

Milestone 4: DELETE ACL Development

Activity 4.3: Creation of Access Control Lists (DELETE ACL)

51. Click New to create a new ACL (Delete).
52. Set Type: record.
53. Set Operation: Delete.
54. Set Name: u_institution_details.
55. Set Active: true.
56. In the Requires role related list, add bb4.
57. No Data condition is required.
58. Save the ACL Record.

The DELETE ACL completes the four-operation access-control implementation.

Activity 4.3: DELETE ACL Verification

Verify:

- Users with bb1, bb2, bb3 and bb4 roles can see, create and edit along with delete the EEE branch records.

The verification confirms the cumulative effect of the four custom roles across READ, CREATE, WRITE and DELETE operations.

The project outcome is a record-level security mechanism in which user roles and record field values are evaluated together to make access decisions.

Milestone 5: Testing and Verification

Milestone 5 focuses on testing the configured ACLs through ServiceNow user impersonation and list access. The objective is to confirm that each ACL operation behaves according to the project configuration.

Activity 5.1: Test READ Access

59. Impersonate the EEE User.
60. Open the u_institution_details list.
61. Check that the intended EEE branch records are visible.
62. Test a user without the required role and verify that records are not accessible.
63. Impersonate Admin and verify that all records can be viewed.

Activity 5.2: Test CREATE, WRITE and DELETE Access

64. Use a user with bb1 and bb2 to verify the New Button / CREATE capability.
65. Use a user with bb1, bb2 and bb3 to verify editing / WRITE capability.
66. Use a user with bb1, bb2, bb3 and bb4 to verify DELETE capability.

Milestone 5: Access-Control Verification Matrix

User / Roles	READ	CREATE	WRITE	DELETE
No required role	No	No	No	No
bb1	Yes – configured EEE access	No	No	No
bb1 + bb2	Yes	Yes	No	No
bb1 + bb2 + bb3	Yes	Yes	Yes	No
bb1 + bb2 + bb3 + bb4	Yes	Yes	Yes	Yes
Admin	Full access	Full access	Full access	Full access

Note: The matrix summarizes the role progression explicitly described by the source project and its verification statements.

Testing should be performed using the ServiceNow instance and the exact ACL configuration created in the previous milestones.

Milestone 5: ServiceNow Execution and Access

The project is executed inside the ServiceNow instance rather than through a separate local application. The main execution flow is:

- Administrator logs in to ServiceNow.
- Users and custom roles are created.
- Institution Details table is configured.
- Student records with ECE, EEE and CSE values are created.
- Security_admin is elevated for ACL configuration.
- READ, CREATE, WRITE and DELETE ACLs are created.
- Users are impersonated to test the configured access.

The source project specifies u_institution_details.list/LIST as the list route used to access the Student Records list during verification.

Access-control behavior is evaluated at the ServiceNow record-security layer.

Exploring the ServiceNow Project Pages

User and Role Management Page:

The User Administration area is used to create the EEE User and the custom roles bb1, bb2, bb3 and bb4. The project assigns the roles to the EEE User for testing.

Table Configuration Page:

System Definition → Tables is used to create the Institution Details table named u_institution_details and to add the required student record fields.

ACL Configuration Page:

System Security → Access Control (ACL) is used to create and configure the READ, CREATE, WRITE and DELETE ACLs. Security_admin must be elevated before ACL configuration.

Student Records List:

The u_institution_details.list/LIST path is used in the source project to open the Student Records list for verification.

Input, Security Rules and Verification

Record Data:

The custom table contains Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number and Description. Branch is configured as a Choice field with ECE, EEE and CSE.

Security Inputs:

- Requested operation: read, create, write or delete.
- Required role: bb1, bb2, bb3 or bb4 depending on the operation.
- READ data condition: Branch is EEE.
- Advanced READ script: grants administrator access, permits bb1, and denies other users.

Verification Results:

- bb1 user: can view the configured EEE branch records.
- User without the required role: cannot view records.
- Admin: can view all records regardless of branch.
- bb1 + bb2: can use the New Button for CREATE.
- bb1 + bb2 + bb3: can view and edit the EEE branch records.
- bb1 + bb2 + bb3 + bb4: can see, create, edit and delete the EEE branch records.

Conclusion

By completing this lab, learners gain a comprehensive understanding of how script-controlled READ, WRITE, CREATE, and DELETE ACLs enforce record-level security, how user roles and record field values are evaluated together to make access decisions, and how ACLs protect sensitive data from unauthorized viewing, modification, creation, and deletion across forms, lists, and Service Portal interfaces.

The project demonstrates a complete access-control workflow in ServiceNow: users and roles are created first, the protected custom table is configured, records with different Branch values are added, and ACL rules are applied to control the four major record operations.

The final verification uses user impersonation and the u_institution_details list to confirm that the configured roles and ACL conditions produce the access behavior described in the project.

End of Project Documentation